

Modül 01 – OOP, SOLID ve UML

İçindekiler

1	Öğrenme çıktıları	2
2	Motivasyon	2
3	OOP'nin temel ilkeleri, bağlaşım ve uyum	3
3.1	Dört temel ilke	3
3.2	Bağlaşım ve uyum	3
4	Mermaid ile UML	3
4.1	Sınıf diyagramları	3
4.2	Sıralama diyagramları	4
5	Tek Sorumluluk İlkesi (SRP)	4
5.1	Problem	4
5.2	İlke	5
5.3	Yapı	5
5.4	Sonra	5
5.5	İkinci örnek: not defteri	6
5.6	Gerçek dünyada kullanımı	6
5.7	Tuzaklar ve ne zaman UYGULANMAMALI	6
6	Açık/Kapalı İlkesi (OCP)	7
6.1	Problem	7
6.2	İlke	7
6.3	Yapı	7
6.4	Sonra	7
6.5	İkinci örnek: dersleri sıralamak	8
6.6	Ara not: sealed tipler bilerek kapalıdır	8
6.7	Gerçek dünyada kullanımı	9
6.8	Tuzaklar ve ne zaman UYGULANMAMALI	9
7	Liskov Yerine Geçme İlkesi (LSP)	9
7.1	Problem	9
7.2	İlke	10
7.3	Yapı	10
7.4	Sonra	10
7.5	İkinci örnek: banka hesapları	10
7.6	Gerçek dünyada kullanımı	11
7.7	Tuzaklar ve ne zaman UYGULANMAMALI	11
8	Arayüz Ayrımı İlkesi (ISP)	11
8.1	Problem	11
8.2	İlke	11
8.3	Yapı	12
8.4	Sonra	12
8.5	İkinci örnek: ürün deposu	12
8.6	Gerçek dünyada kullanımı	13
8.7	Tuzaklar ve ne zaman UYGULANMAMALI	13
9	Bağımlılığın Tersine Çevrilmesi İlkesi (DIP)	13

9.1	Problem	13
9.2	İlke	13
9.3	Yapı	13
9.4	Sonra	14
9.5	İkinci örnek: bir bağımlılık olarak zaman	14
9.6	Gerçek dünyada kullanımı	15
9.7	Tuzaklar ve ne zaman UYGULANMAMALI	15
10	Kalıtım yerine bileşim	15
10.1	Problem	15
10.2	İlke	15
10.3	Yapı	16
10.4	Sonra	16
10.5	İkinci örnek: araçlar	16
10.6	Gerçek dünyada kullanımı	17
10.7	Tuzaklar ve ne zaman UYGULANMAMALI	17
11	GoF kataloğu	17
12	Özet	17
13	Sınav	18
14	Ödevler	18
15	İleri okuma	18

2. Hafta · Ön koşullar: m00 (record'lar, sealed arayüzler, switch desenleri, lambdalar) · Tahmini çalışma süresi: 5 saat

Her örnek derlemeden çalıştırın:

```
java modules/m01-oop-solid-uml/src/main/java/io/github/aliturgutbozkurt/patterns/m01/examples/<yol>/<Demo>.java
```

(JDK 27)

1 Öğrenme çıktıları

Bu modülün sonunda şunları yapabilirsiniz:

1. OOP'nin dört temel ilkesini **açıklamak** ve bir sınıfın bağlaşımını (coupling) ve uyumunu (cohesion) **değerlendirmek**.
2. Her SOLID ilkesinin ihlalini **tanımak** ve onu, bir karakterizasyon testiyle korunarak, davranışı değiştirmeden **yeniden düzenlemek**.
3. Kalıtım ile bileşim arasında **karar vermek** ve yönlendiren (forwarding) bir sarmalayıcı **yazmak**.
4. Bağımlılıkları — `java.time.Clock` ile zamanı da — dışarıdan **vermek**; böylece kodu yan etki olmadan test etmek.
5. Mermaid ile UML sınıf ve sıralama diyagramları **çizmek** ve dersin geri kalanındaki diyagramları **okumak**.
6. GoF kataloğunu **tanımlamak** ve her kalıbın bu derste nerede işlendiğini bulmak.

2 Motivasyon

Bu derste her tasarım kalıbı aynı birkaç fikirle gerekçelendirilir: *bu sınıfın değişmek için çok fazla nedeni var, her yeni durum için bu kodun değiştirilmesi gerekiyor, bu alt sınıf onu kullananları şaşırtıyor, bu istemci hiç kullanmadığı metotlara bağımlı, bu iş kuralı bir ayrıntıya kaynaklanmış*. Bu beş cümle SOLID ilkeleridir. Onları kodda duymayı öğrenirseniz kalıplar ezberlenecek tarifler olmaktan çıkar — açık çözüm hâline gelir.

Aşağıdaki her ilke aynı şekilde işlenir: sorunu gösteren küçük bir **önce** sürümü, onu düzelten bir **sonra** sürümü ve ikisinin aynı davrandığını kanıtlayan bir test. Her demo iki sürümü de yazdırır.

3 OOP'nin temel ilkeleri, bağlaşım ve uyum

3.1 Dört temel ilke

İlke	Anlamı	Bu modüldeki örnek
Kapsülleme	Durumu gizle; nesne kendi değişmezlerini (invariant) korusun	<code>CheckingAccount</code> bakiyesinin asla eksiye düşmesine izin vermez
Soyutlama	Nesnenin <i>ne</i> yaptığını göster, <i>nasıl</i> yaptığını değil	<code>MessageSender</code> "mesaj gönder" der, "SMTP soketi aç" demez
Kalıtım	Alt tip, üst tipi yeniden kullanır ve genişletir	<code>Square</code> extends <code>Rectangle</code> — ve nasıl ters gittiği
Çok biçimlilik	Tek çağrı, çalışma zamanında seçilen bir çok davranış	<code>PrintQueue</code> her <code>Printer</code> 'da, bir lambda-da bile yazdırır

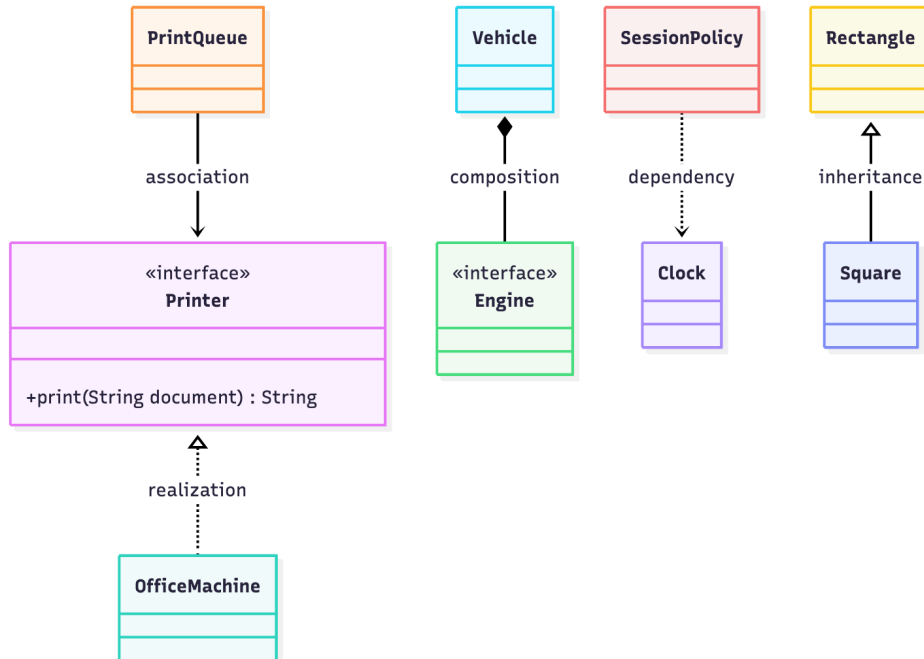
3.2 Bağlaşım ve uyum

Uyum (cohesion), bir sınıfın parçalarının birbirine ne kadar ait olduğudur. **Bağlaşım (coupling)**, bir sınıfın bir başkası hakkında ne kadar şey bildiğidir. İyi tasarımlarda *uyum yüksek, bağlaşım düşüktür (gevşektir)*: her sınıf tek bir işi iyi yapar ve içindeki bir değişiklik sisteme yayılmaz. Her SOLID ilkesi, uyumu artırmanın ya da bağlaşımı azaltmanın bir yoludur.

4 Mermaid ile UML

Bu derste diyagramlar [Mermaid](#) ile metin olarak çizilir; böylece kodun yanında yaşarlar, GitHub'da görünülürler ve PDF'lerde önceden işlenirler.

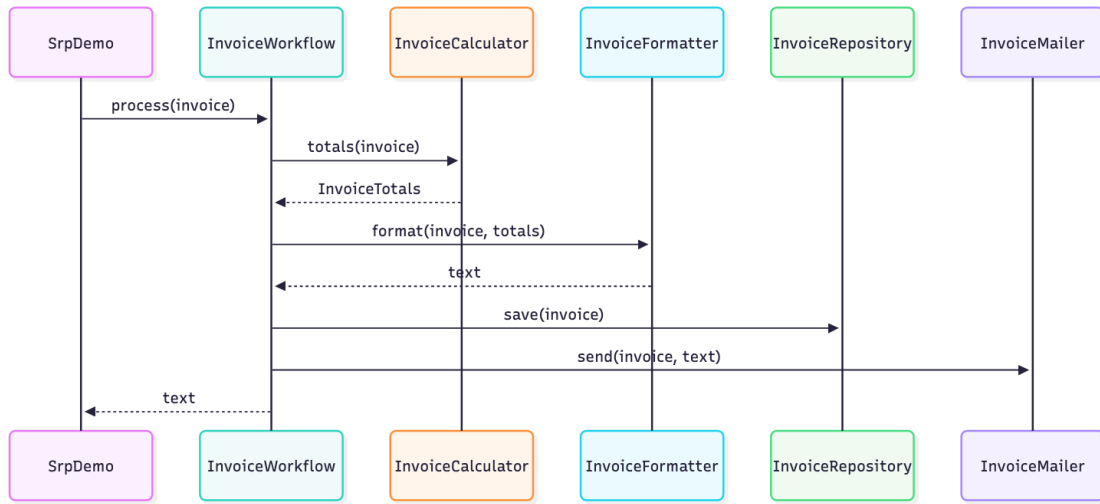
4.1 Sınıf diyagramları



Ok	Mermaid	Okunuşu
Kalıtım (inheritance)	A < -- B	B bir A'dır (extends)
Gerçekleştirme (realization)	A < .. B	B, A arayüzünü uygular
İlişki (association)	A --> B	A, B'ye bir referans tutar
Toplama (aggregation)	A o-- B	A'nın B'si vardır ama B, A olmadan da yaşar
Bileşim (composition)	A *-- B	A, B'nin sahibidir; B'nin ömrü A'nın ömrüdür
Bağımlılık (dependency)	A ..> B	A, B'yi kısa süreliğine kullanır (parametre, yerel değişken)

4.2 Sıralama diyagramları

Sıralama diyagramı *kimin kimi, hangi sırayla çağırdığını* gösterir. Zaman yukarıdan aşağı akar:



5 Tek Sorumluluk İlkesi (SRP)

5.1 Problem

`InvoiceService` toplamları hesaplar, faturayı biçimlendirir, saklar ve e-postayla gönderir. Yeni bir vergi kuralı, yeni bir yerleşim, bir veritabanı ve yeni bir e-posta sağlayıcısı; aynı sınıfı açmak için birbiriyle ilgisiz dört neden – ve diğerlerini bozmanın dört yolu.

```

// file: examples/srp/before/InvoiceService.java
public final class InvoiceService {
    // ...
    /** Calculates, formats, stores and "sends" the invoice; returns its text. */
    public String process(String number, String customer, List<Line> lines) {
        // ...
        // 1) calculation
        BigDecimal subtotal = BigDecimal.ZERO;
        // ...
        // 2) formatting
        var text = new StringBuilder();
        // ...
        // 3) storage
        sentInvoices.put(number, result);
        // 4) delivery
    }
}
  
```

```

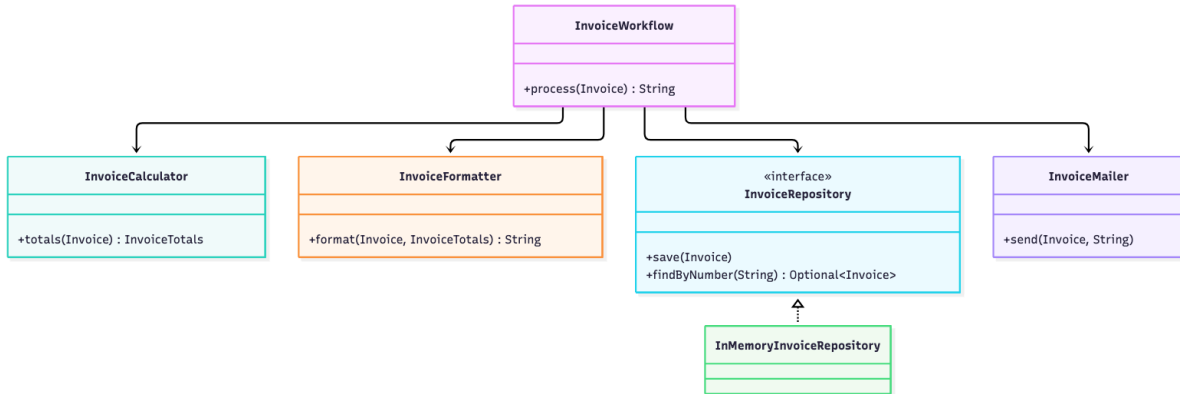
        System.out.println("Emailing invoice " + number + " to " + customer);
        return result;
    }

```

5.2 İlke

Bir sınıfın **değişmek için tek bir nedeni** olmalıdır — tek bir aktöre (vergi dairesi, tasarımcı, veritabanı yöneticisi, ...) hesap vermelidir.

5.3 Yapı



5.4 Sonra

Artık her iş birlikçinin tek bir görevi var. İş akışı yalnızca adımların *sırasını* belirler:

```

// file: examples/srp/after/InvoiceWorkflow.java
public final class InvoiceWorkflow {
    // ...
    /** Calculates, formats, stores and sends the invoice; returns its text. */
    public String process(Invoice invoice) {
        String text = formatter.format(invoice, calculator.totals(invoice));
        repository.save(invoice);
        mailer.send(invoice, text);
        return text;
    }
}

```

Para kuralları, bir vergi değişikliğinin — ve yalnızca bir vergi değişikliğinin — dokunduğu küçük bir sınıfta yaşar:

```

// file: examples/srp/after/InvoiceCalculator.java
public final class InvoiceCalculator {

    /** 20 % VAT (Turkish KDV). */
    public static final BigDecimal VAT_RATE = new BigDecimal("0.20");

    public InvoiceTotals totals(Invoice invoice) {
        BigDecimal subtotal = invoice.lines().stream()
            .map(InvoiceLine::amount)
            .reduce(BigDecimal.ZERO, BigDecimal::add)
            .setScale(2, RoundingMode.HALF_EVEN);
        BigDecimal vat = subtotal.multiply(VAT_RATE).setScale(2,

```

```

RoundingMode.HALF_EVEN);
    return new InvoiceTotals(subtotal, vat, subtotal.add(vat));
}
}

```

Yeniden düzenlemenin hiçbir şeyi değiştirmedigini nereden biliyoruz? `InvoiceTest.refactoredWorkflowBehavesExactlyLikeTheGodClass` iki sürümü de çalıştırır ve çıktıları karşılaştırır. Yeniden düzenlemeden önce *mevcut* davranışı sabitleyen teste **karakterizasyon testi** denir. `SrpDemo` iki sürümü de yazdırır:

```

INVOICE INV-001
Customer: Ada Lovelace
  2 x Keyboard      49.90    99.80
  1 x Mouse         19.99    19.99
  1 x Monitor       229.00   229.00
Subtotal:      348.79
VAT 20%:       69.76
Total:         418.55
same text? true

```

5.5 İkinci örnek: not defteri

`GradeBook.report` CSV satırlarını ayrıştırır, ortalamaları hesaplar, harf notlarına çevirir ve bir tablo yazdırır — tek bir metotta. Yeniden düzenleme **girdi biçimini** (`ScoreParser`), **notlandırma politikasını** (`GradingScale`) ve **yerleşimi** (`GradeReport`) ayırır. Fakülte bir not sınırını kaydirdığında yalnızca bu sınıf değişir:

```

// file: examples/srp/gradebook/after/GradingScale.java
/** Letter grade for an average in {@code [0, 100]}. */
public String letterFor(BigDecimal average) {
    if (average.signum() < 0 || average.compareTo(BigDecimal.valueOf(100)) > 0) {
        throw new IllegalArgumentException("average out of range 0-100: " +
average);
    }
    return BOUNDARIES.stream()
        .filter(boundary -> average.compareTo(boundary.minimum()) >= 0)
        .map(Boundary::letter)
        .findFirst()
        .orElse("FF");
}
}

```

```

// file: examples/srp/GradeBookDemo.java
String after = new GradeReport(new GradingScale()).render(new
ScoreParser().parse(CSV));

```

5.6 Gerçek dünyada kullanımı

`java.time`; değerleri (`LocalDate`), biçimlendirmeyi (`DateTimeFormatter`) ve zaman kaynaklarını (`Clock`) ayrı tiplerde tutar. JDBC; bağlanmayı (`DataSource`), bir komut çalıştırmayı (`PreparedStatement`) ve satırları okumayı (`ResultSet`) ayırır.

5.7 Tuzaklar ve ne zaman UYGULANMAMALI

- "Değişmek için tek neden", "tek metot" demek değildir. Her metodu ayrı bir sınıfa bölmek uyumu yeniden düşürür.

- İki sorumluluk gerçekten farklı zamanlarda ya da farklı kişiler için değiştiğinde bölün — önceden değil.

6 Açık/Kapalı İlkesi (OCP)

6.1 Problem

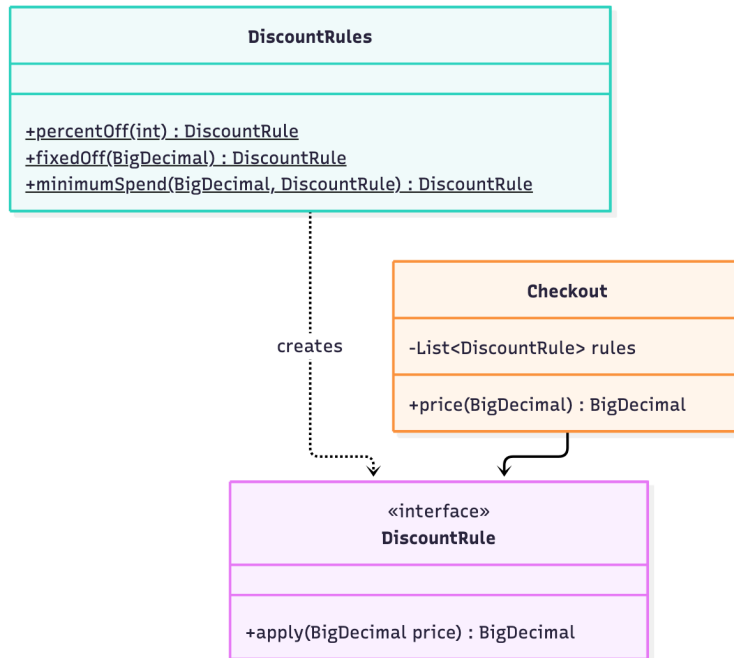
Her yeni indirim türü, `PriceCalculator`'ı açıp bir dal daha eklemek — ve eski dalların hepsini yeniden test etmek — demektir:

```
// file: examples/ocp/before/PriceCalculator.java
BigDecimal percentOff;
if (customerType.equals("REGULAR")) {
    percentOff = BigDecimal.ZERO;
} else if (customerType.equals("STUDENT")) {
    percentOff = BigDecimal.valueOf(10);
} else if (customerType.equals("VIP")) {
    percentOff = BigDecimal.valueOf(15);
} else {
    throw new IllegalArgumentException("unknown customer type: " +
customerType);
}
```

6.2 İlke

Yazılım birimleri **genişletmeye açık, değiştirmeye kapalı** olmalıdır: yeni davranış, çalışan kodu düzenleyerek değil, yeni kod olarak eklenir.

6.3 Yapı



6.4 Sonra

Genişleme noktası fonksiyonel bir arayüzdür; bu yüzden yeni bir kural bir lambda olabilir:

```
// file: examples/ocp/after/DiscountRule.java
@FunctionalInterface
public interface DiscountRule {

    /** Returns the price after this discount. */
    BigDecimal apply(BigDecimal price);
}
```

`Checkout` kendisine hangi kurallar verildiyse onları uygular ve hangilerinin var olduğunu bilmesi gerekmez:

```
// file: examples/ocp/after/Checkout.java
public BigDecimal price(BigDecimal amount) {
    if (amount.signum() < 0) {
        throw new IllegalArgumentException("amount must not be negative: " +
amount);
    }
    BigDecimal price = amount;
    for (DiscountRule rule : rules) {
        price = rule.apply(price);
    }
    return price.max(BigDecimal.ZERO).setScale(2, RoundingMode.HALF_EVEN);
}
```

Bir "Black Friday" kuralı `OcpDemo`'da iki satır olarak gelir — `Checkout`'a dokunulmaz:

```
// file: examples/ocp/OcpDemo.java
DiscountRule roundDownToWholeLira = price -> price.setScale(0,
RoundingMode.FLOOR);
var blackFriday = new Checkout(List.of(percentOff(15), roundDownToWholeLira));
```

6.5 İkinci örnek: dersleri sıralamak

`CourseSorter` bir sıralama anahtarı dizgesi üzerinde `switch` yapar. JDK bu sorunu zaten çözmüştür: `Comparator` bir genişleme noktasıdır; `comparing`, `thenComparing` ve `reversed` eski sıralamalardan yenilerini oluşturur. `CourseCatalog` her `Comparator`'ı kabul eder:

```
// file: examples/ocp/CourseSortDemo.java
Comparator<Course> byDepartmentThenCredits = comparing(Course::department)
    .thenComparing(comparingInt(Course::credits).reversed())
    .thenComparing(Course::code);
```

6.6 Ara not: sealed tipler bilerek kapalıdır

Eksiksiz bir `switch` ile birlikte kullanılan `sealed` bir arayüz, *tersi* bir ödüneşimdir. Yeni bir `Engine` eklemek, onun üzerindeki her `switch`'i düzenlemenizi gerektirir ve derleyici her birini listeler:

```
// file: examples/composition/vehicles/after/Vehicle.java
double range = switch (engine) {
    case Engine.Petrol(double tank, double per100) -> tank * 100 / per100;
    case Engine.Diesel(double tank, double per100) -> tank * 100 / per100;
    case Engine.Electric(double battery, double per100) -> battery * 100 /
per100;
};
```

Buna *ifade problemi* (*expression problem*) denir. Açık bir arayüz yeni **tipleri** ucuz, yeni **işlemleri** pahalı kılar; `switch` ile kullanılan sealed bir tip yeni **işlemleri** ucuz, yeni **tipleri** pahalı kılar. Hangisini daha sık ekleyeceğinizi düşünerek seçin. m08 modülü (Visitor ve desen eşleme) bu konuya geri döner.

6.7 Gerçek dünyada kullanımı

`Comparator`, `Collector`, `java.util.function` ve `ServiceLoader` (m02) birer genişleme noktasıdır: JDK değiştirmeye kapalıdır, ama siz onun davranışını her gün genişletirsiniz.

6.8 Tuzaklar ve ne zaman UYGULANMAMALI

- "Belki lazım olur" diye genişleme noktası eklemeyin. İlk farklılık bir `if` olabilir; ikinci ya da üçüncüsü geldiğinde bir kural arayüzü çıkarın.
- Sırayla uygulanan kurallar birbirini etkiler: `appliesRulesInOrder` testleri, önce %10 sonra -20'nin, önce -20 sonra %10 ile aynı olmadığını gösterir.

7 Liskov Yerine Geçme İlkesi (LSP)

7.1 Problem

Geometride kare bir dikdörtgendir. Değiştirilebilir bir alt tip olarak ise değildir: kenarlarını eşit tutmak için `Square` her setter'da iki kenarı birden değiştirmek zorundadır.

```
// file: examples/lsp/before/Square.java
public final class Square extends Rectangle {

    public Square(double side) {
        // Flexible constructor body (JEP 513): validate with Square's own message
        // before super(...) runs.
        if (!(side > 0)) {
            throw new IllegalArgumentException("side must be positive: " + side);
        }
        super(side, side);
    }

    @Override
    public void setWidth(double width) {
        super.setWidth(width);
        super.setHeight(width);
    }

    // ...
}
```

`Rectangle`'a göre yazılmış bir istemci haklı olarak 20 bekler — `Square` için 16 alır:

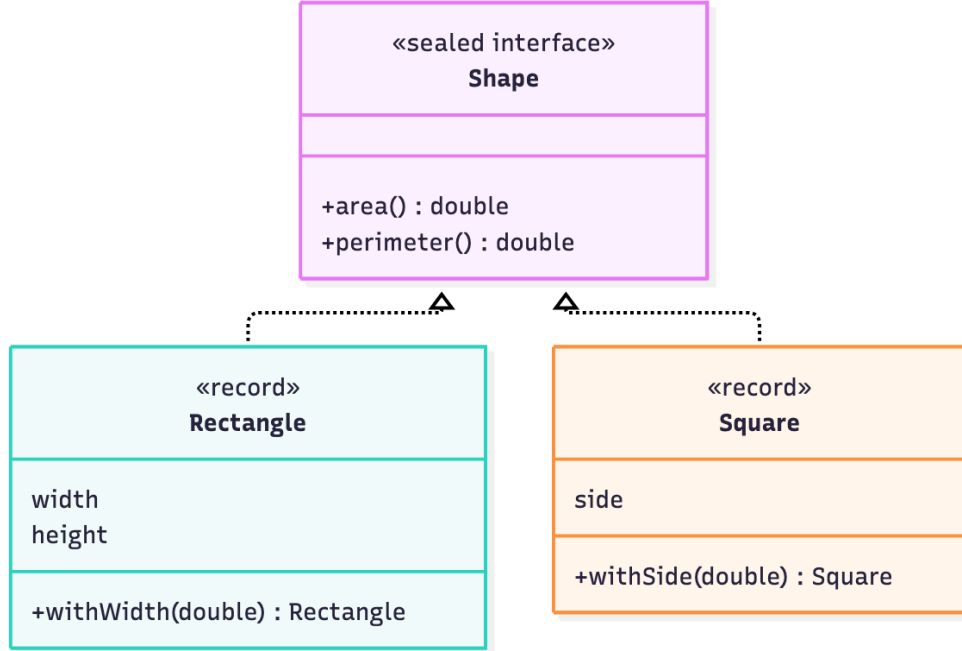
```
// file: examples/lsp/before/RectangleClient.java
/** Resizes to 5 × 4 and returns the area – the client reasonably expects 20. */
public static double resizeTo5By4(Rectangle rectangle) {
    rectangle.setWidth(5);
    rectangle.setHeight(4);
    return rectangle.area();
}
```

Kurucuya dikkat edin: Java 25'ten beri (JEP 513) `super(...)` çağrısından **önce** komut çalıştırılabilir; böylece `Square`, `Rectangle` kurucusu çalışmadan önce kenarını kendi hata mesajıyla doğrular.

7.2 İlke

Bir alt tipin nesneleri, **üst tipin beklendiği her yerde**, istemci fark etmeden kullanılabilir. Alt tip ön koşulları güçlendiremez, son koşulları zayıflatamaz ve üst tipin değişmezlerini bozamaz.

7.3 Yapı



7.4 Sonra

Değişmez record'ların setter'ı yoktur; dolayısıyla bozulacak bir setter sözleşmesi de yoktur. `Square` ve `Rectangle` kardeş olur:

```
// file: examples/lsp/after/Rectangle.java
public record Rectangle(double width, double height) implements Shape {
    // ...
    public Rectangle withWidth(double newWidth) {
        return new Rectangle(newWidth, height);
    }
}
```

7.5 İkinci örnek: banka hesapları

`FixedDepositAccount` extends `Account` her para çekme isteğini reddeder — `withdraw`'un **ön koşulu**nu **güçlendirir**: "tutar ≤ bakiye" koşulunu "asla" yapar:

```
// file: examples/lsp/accounts/before/FixedDepositAccount.java
@Override
public void withdraw(BigDecimal amount) {
    throw new UnsupportedOperationException("no withdrawals from a fixed deposit");
}
```

Çözüm, yeteneği reddetmek yerine modellemektir. `Account` yalnızca her hesabın yapabildiğini vaat eder; `Withdrawable` ayrı bir roldür ve `BillPayer` tam olarak bu rolü ister:

```
// file: examples/lsp/accounts/after/BillPayer.java
/** Pays a bill and returns the remaining balance. */
public static BigDecimal pay(Withdrawable source, BigDecimal bill) {
    source.withdraw(bill);
    return source.balance();
}
```

pay'e bir `FixedDepositAccount` vermek artık çalışma zamanında bir sürpriz değil, derleme hatasıdır. Bir arayüzü istemcilerin ihtiyacına göre bölmek bir sonraki ilkedir.

7.6 Gerçek dünyada kullanımı

`List.of(...)`, `add` metodu `UnsupportedOperationException` fırlatan bir `List` döndürür. JDK bu metotları "isteğe bağlı işlemler (optional operations)" olarak belgeler — pragmatik bir ödünleşim, ama LSP'nin uyardığı sürprizin ta kendisi.

7.7 Tuzaklar ve ne zaman UYGULANMAMALI

- Gerçek dünyadaki "bir ...dır" ilişkisi yetmez; üst tipin davranışının **yerine geçebiliyor mu** diye sorun.
- Üst tipe göre yazılmış bir test takımı (ödevlerdeki sözleşme testleri gibi) en iyi LSP kontrolüdür: her uygulama onu geçmelidir.

8 Arayüz Ayrımı İlkesi (ISP)

8.1 Problem

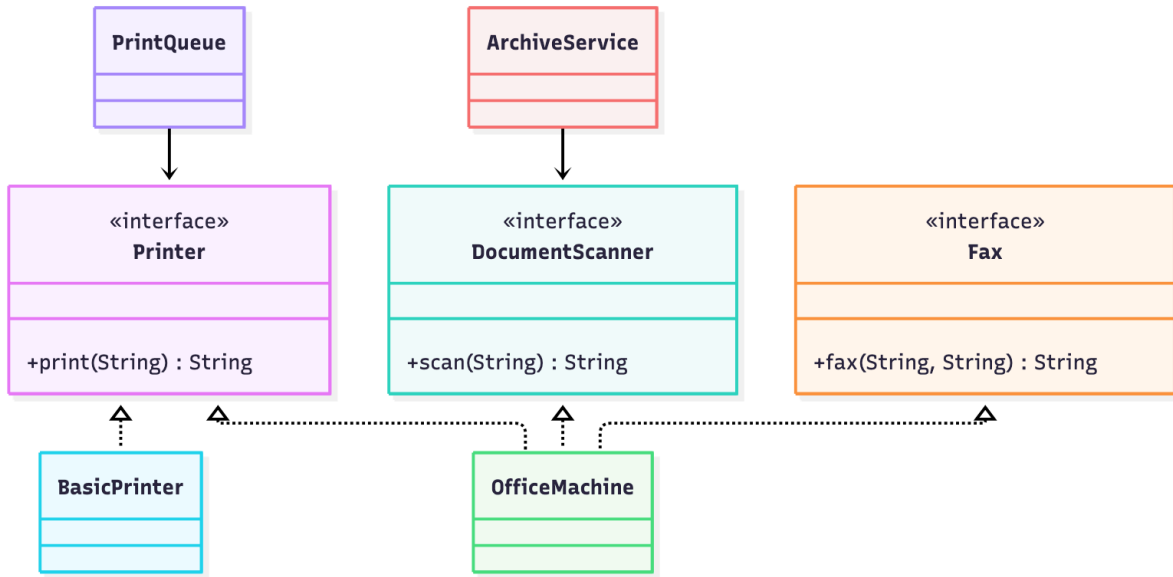
`MultiFunctionDevice` her cihazı yazdırmaya, taramaya ve faks göndermeye zorlar. Basit bir yazıcı yalnızca hata fırlatabilir:

```
// file: examples/isp/before/BasicPrinter.java
@Override
public String scan(String page) {
    throw new UnsupportedOperationException("BasicPrinter cannot scan");
}
```

8.2 İlke

İstemciler kullanmadıkları metotlara bağımlı olmaya zorlanmamalıdır. Tek büyük arayüz yerine birkaç küçük **rol arayüzü** tercih edin.

8.3 Yapı



8.4 Sonra

Her rol küçük bir arayüzdür — lambda olabilecek kadar küçük. Bir sınıf yine de birkaç rol oynayabilir:

```
// file: examples/isp/after/Printer.java
@FunctionalInterface
public interface Printer {

    String print(String document);
}
```

```
// file: examples/isp/after/OfficeMachine.java
public final class OfficeMachine implements Printer, DocumentScanner, Fax {
```

İstemci ihtiyaç duyduğu en küçük rolü adıyla ister:

```
// file: examples/isp/after/PrintQueue.java
public PrintQueue(Printer printer) {
    this.printer = Objects.requireNonNull(printer, "printer");
}
```

8.5 İkinci örnek: ürün deposu

Beş metotlu bir `ProductStore`'un salt okunur görünümü `save`, `delete` ve `importAll` metotlarını hata fırlatarak "uygulamak" zorundadır. Bölmeden sonra `CatalogPage` yalnızca `ProductReader`'a bağlıdır; yazma metotlarını göremez bile. `InMemoryProducts` iki rolü de uygular. Bir varsayılan (default) metot yazma rolünü kullanışlı tutar:

```
// file: examples/isp/store/after/ProductWriter.java
/** Saves every product in order. */
default void importAll(List<Product> products) {
    products.forEach(this::save);
}
```

8.6 Gerçek dünyada kullanımı

JDK rol arayüzleriyle doludur: `Readable`, `Appendable`, `AutoCloseable`, `Comparable`, `Iterable`. Hem `StringBuilder` hem `Writer` birer `Appendable`'dir; yalnızca ekleme yapan kod ikisiyle de çalışır.

8.7 Tuzaklar ve ne zaman UYGULANMAMALI

- İstemcilerin birlikte kullandığı metotları ayırmayın; her yerde arayüz başına tek metot, bağlantı kurmayı zorlaştırır.
- İstemci ihtiyacına göre ayırın, uygulama ayrıntısına göre değil.

9 Bağımlılığın Tersine Çevrilmesi İlkesi (DIP)

9.1 Problem

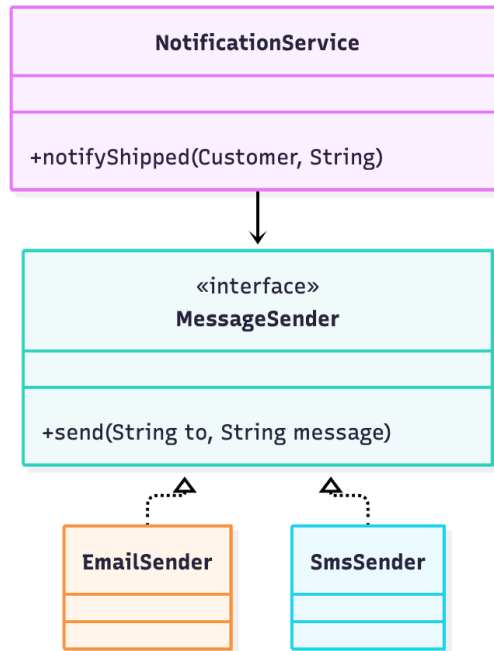
"Müşteriye siparişinin kargoya verildiğini bildir" iş kuralı somut bir e-posta göndericisini kendisi oluşturur. SMS gönderemez ve gerçekten "göndermeden" test edilemez:

```
// file: examples/dip/before/NotificationService.java
public void notifyShipped(Customer customer, String orderId) {
    var sender = new EmailSender(); // hard-wired dependency on a concrete class
    sender.send(customer.email(), "Order " + orderId + " has shipped, " +
customer.name() + ".");
}
```

9.2 İlke

Üst düzey politika alt düzey ayrıntılara bağımlı olmamalıdır; **ikisi de soyutlamalara bağımlıdır** — ve soyutlamanın sahibi üst düzey taraftır.

9.3 Yapı



`EmailSender` ve `SmsSender`'dan çıkan oklar artık `NotificationService`'in yanında duran bir arayüze, *yukarı* bakıyor — "tersine çevirme" budur.

9.4 Sonra

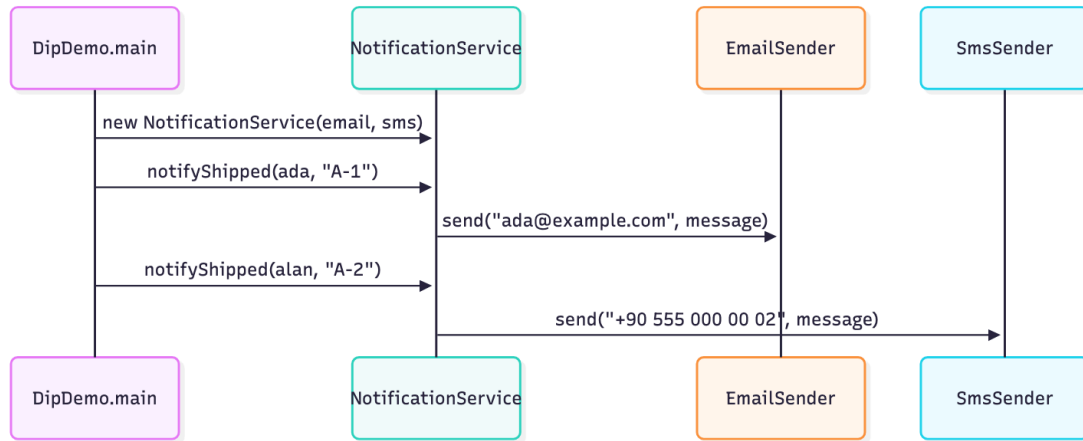
```
// file: examples/dip/after/NotificationService.java
public NotificationService(MessageSender email, MessageSender sms) {
    this.email = Objects.requireNonNull(email, "email");
    this.sms = Objects.requireNonNull(sms, "sms");
}

public void notifyShipped(Customer customer, String orderId) {
    String message = "Order " + orderId + " has shipped, " + customer.name() + ".";
    switch (customer.preferred()) {
        case EMAIL -> email.send(customer.email(), message);
        case SMS -> sms.send(customer.phone(), message);
    }
}
```

Yine de birinin `new` demesi gerekir. O yer **bileşim kökü (composition root)** — burada `main`:

```
// file: examples/dip/DipDemo.java
var service = new NotificationService(new EmailSender(), new SmsSender());
```

Çalışma zamanındaki çağrı sırası:



Testlerde, mesajları kaydeden bir `MessageSender` (bir **test ikizi**) gerçek göndericilerin yerini alır — konsol çıktısını yakalamaya gerek kalmaz.

9.5 İkinci örnek: bir bağımlılık olarak zaman

İş mantığının içindeki `Instant.now()`, sistem saatine gizli bir bağımlılıktır. JDK'nın kendi soyutlaması `java.time.Clock` 'tur; onu dışarıdan verin, testler zamanı istedikleri ana sabitleyebilirsiniz:

```
// file: examples/dip/clock/SessionPolicy.java
/** Expired from the instant {@code startedAt + timeout} onwards. */
public boolean isExpired(Session session) {
    return !clock.instant().isBefore(session.startedAt().plus(timeout));
}
```

```
session for ada started at 2026-09-29T09:00:00Z, timeout PT30M
at 2026-09-29T09:29:59Z -> active
at 2026-09-29T09:30:00Z -> expired
```

9.6 Gerçek dünyada kullanımı

JDBC kodu belirli bir sürücüye değil `javax.sql.DataSource` 'a bağlıdır; günlükleme kodu bir kütüphaneye değil `System.Logger` 'a bağlıdır. Dependency Injection (Bağımlılık Enjeksiyonu) çatıları bileşim kökünü otomatikleştirir — m11 modülünde bir tanesini elle kuracağız.

9.7 Tuzaklar ve ne zaman UYGULANMAMALI

- Her `new` bir sorun değildir: değerler (`BigDecimal`, record'lar) ve kararlı JDK tipleri doğrudan oluşturulabilir.
- "Test edilebilirlik için" sonsuza dek tek uygulaması olacak her sınıfa bir arayüz eklemek törendir; gerçek bir farklılık ya da yavaş/yan etkili bir bağımlılık olduğunda tersine çevirin.

10 Kalıtım yerine bileşim

10.1 Problem

`CountingSet` extends `HashSet` şimdiye kadar kaç eleman eklendiğini saymak istiyor:

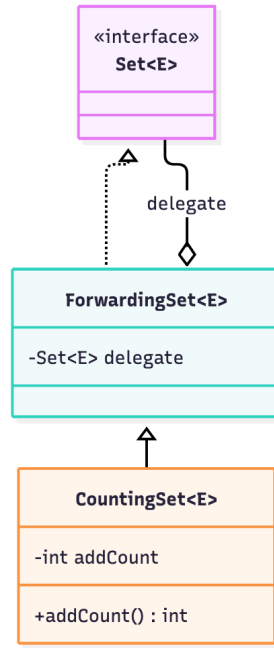
```
// file: examples/composition/before/CountingSet.java
@Override
public boolean addAll(Collection<? extends E> elements) {
    addCount += elements.size();
    return super.addAll(elements); // calls this.add(...) for each element ->
    counted again
}
```

`HashSet.addAll`, her eleman için `add`'i çağırır — bizim ezdiğimiz (override) `add`'i. Üç eleman altı kez sayılır. Alt sınıf, üst sınıfının bir uygulama ayrıntısına bağlıdır: **kırılgan taban sınıf** sorunu.

10.2 İlke

Davranışı yeniden kullanmak istediğinizde "**bir ...dır**" (kalıtım) yerine "**bir ...sı vardır**" (bileşim) tercih edin. Yalnızca kontrol ettiğiniz, gerçek bir alt tip ilişkisi için kalıtım kullanın.

10.3 Yapı



10.4 Sonra

`ForwardingSet`, her çağrıyı sarmaladığı kümeye yönlendirerek `Set`'i uygular. `CountingSet` yönlendiriciyi — kontrol ettiğimiz bir sınıfı — genişletir ve herhangi bir `Set`'i sarmalar:

```
// file: examples/composition/after/ForwardingSet.java
public class ForwardingSet<E> implements Set<E> {

    private final Set<E> delegate;

    public ForwardingSet(Set<E> delegate) {
        this.delegate = Objects.requireNonNull(delegate, "delegate");
    }
    // ...
    @Override public boolean add(E e) { return delegate.add(e); }
    // ...
    @Override public boolean addAll(Collection<? extends E> c) { return
    delegate.addAll(c); }
}
```

Sarmalanan kümenin `addAll`'u bizimkini değil *kendi* `add`'ini çağırır; böylece her eleman bir kez sayılır. Aynı yapı m04'te **Decorator (Dekorator)** olarak geri döner.

10.5 İkinci örnek: araçlar

Kalıtımla her motor $\times$ vites kombinasyonu kendi alt sınıfını ister — $2 \times 2 = 4$ sınıf; bir elektrik motoru 2 tane daha ekler. Bileşimle bir aracın bir motoru ve bir vites kutusu *vardır*; her boyut kendi başına değişir:

```
// file: examples/composition/vehicles/after/Vehicle.java
public record Vehicle(Engine engine, Gearbox gearbox) {
```

Elektrik motorunu eklemek tek bir yeni record gerektirdi; `VehicleDemo` altı kombinasyonun hepsini yazdırır.

10.6 Gerçek dünyada kullanımı

`java.util.Stack` extends `Vector` ve `java.util.Properties` extends `Hashtable` JDK'nın kendi pişmanlıklarıdır: ikisi de amaçlanan kullanımlarını bozan metotları kalıtımla alır. Buna karşılık `Collections.unmodifiableSet(...)` ve `Collections.synchronizedList(...)` birer sarmalayıcıdır — yani bileşim.

10.7 Tuzaklar ve ne zaman UYGULANMAMALI

- Yönlendiren sınıf basmakalıp koddur; sarmalanan tip sizin değilse ya da birçok uygulaması varsa kendini amorti eder.
- Kendi kodunuzda gerçek ve kararlı bir "bir ...dır" ilişkisi için kalıtım hâlâ doğrudur; özellikle her alt tipi kontrol ettiğiniz `sealed` hiyerarşilerde.

11 GoF kataloğu

1994'te Gamma, Helm, Johnson ve Vlissides ("Dörtlü Çete", Gang of Four) üç ailede 23 kalıbı katalogladı. Bu ders hepsini modern Java ile işler:

Aile	Kalıp	Modül
Yaratımsal	Singleton, Factory Method, Abstract Factory	m02
Yaratımsal	Builder, Prototype	m03
Yapısal	Adapter, Decorator, Proxy	m04
Yapısal	Composite, Bridge, Facade, Flyweight	m05
Davranışsal	Strategy, Template Method, Command, Iterator	m06
Davranışsal	Observer, Mediator, Chain of Responsibility, Memento	m07
Davranışsal	State, Visitor, Interpreter	m08

Bunların birkaçıyla bu modülde adını anmadan zaten karşılaştınız: `DiscountRule` bir **Strategy (Strateji)**, `ForwardingSet` bir **Decorator**'in iskeleti, `InvoiceWorkflow` ise bir **Facade (Cephe)**'ye yakındır.

12 Özet

İlke	Belirti	Çözüm	Java 27 kısayolu
SRP	Bir sınıf birbiriyle ilgisiz bir-kaç nedenle değişir	Değişme nedenine göre böl; bir koordinatör tut	Veri için record'lar
OCP	Yeni bir durum çalışan bir sınıfı değiştirir	Bir genişleme noktası (arayüz)	Fonksiyonel arayüz + lambda
LSP	Bir alt tip onu kullananları şaşırtır	Yetenekleri modelle; değişmez değerler	Sealed record'lar, setter yok
ISP	Uygulayıcılar <code>UnsupportedOperationException</code> fırlatır	Küçük rol arayüzleri	<code>@FunctionalInterface</code> roller
DIP	Politika bir ayrıntıya <code>new</code> der	Politikanın sahip olduğu bir soyutlamayı ver	<code>java.time.Clock</code> , test ikizi olarak lambdalar
Bileşim	Alt sınıf üst sınıfın iç yapısına bağlı	Sarmala ve yönlendir	Sealed parçalardan oluşan record'lar

13 Sınav

1. `InvoiceFormatter` ve `InvoiceCalculator`'ın ikisini de `InvoiceWorkflow` kullanıyor. Her birini hangi aktör değiştirir?
2. Karakterizasyon testi nedir ve neden yeniden düzenlemeden önce yazılır?
3. Eksiksiz bir `switch` ile kullanılan `sealed` bir arayüz neden OCP'yi zararlı bir şekilde ihlal etmez?
4. `Square extends Rectangle` LSP'yi bozar. İki sınıf da değişmez olsaydı yine bozar mıydı? Neden?
5. `FixedDepositAccount.withdraw` hangi LSP kuralını bozar: ön koşulları, son koşulları mı, değişmezleri mi?
6. ISP'ye uygun bir JDK rol arayüzü ve bir "isteğe bağlı işlem" örneği verin.
7. DIP'te `MessageSender` arayüzünün sahibi kimdir ve somut göndericiler nerede oluşturulur?
8. `CountingSet extends HashSet` neden `addAll`'u iki kez sayar ve yönlendiren sürümde bu hata neden olamaz?
9. Şunların Mermaid okunu çizin: "`Vehicle` bir `Engine`'e sahiptir" ve "`SessionPolicy` kendisine verilen bir `Clock`'u kullanır".

Cevaplar

1. Vergi dairesi (KDV kuralları) hesaplayıcıyı değiştirir; fatura yerleşimini tasarlayan kişi biçimlendiriciyi değiştirir.
2. Kodun *bugün* ne yaptığını kaydeden bir testtir. Yeniden düzenlemeyi güvenli kılar: çıktı değişirse test başarısız olur.
3. Bilinçli bir seçimdir: yeni işlemler kolaydır ve derleyici, yeni bir *tipin* ele alınması gereken her yeri listeler. OCP, değişmesini beklediğiniz eksenle ilgilidir.
4. Hayır. Setter olmadan "yalnızca genişliği değiştir" sözleşmesi yoktur; `setWidth` yeni bir değer döndürür.
5. Ön koşulu güçlendirir: `Account`'un ön koşulunu ($\text{tutar} \leq \text{bakiye}$) sağlayan çağrılar yine de başarısız olur.
6. `Appendable` / `AutoCloseable` / `Comparable`; `List.of(...).add(...)` isteğe bağlı bir işlem için hata fırlatır.
7. Sahibi üst düzey taraftır (`NotificationService`'in yanında durur); göndericiler bileşim kökü olan `main`'de oluşturulur.
8. `HashSet.addAll`, ezilmiş `add`'i çağırır. Yönlendiren sürüm `delegate.addAll`'u çağırır; o da bizimkini değil, sarmalanan kümenin kendi `add`'ini çağırır.
9. `Vehicle` `Engine` (bileşim) ve `SessionPolicy` `Clock` (bağımlılık; buradaki gibi bir alanda tutuluyorsa `-->`).

14 Ödevler

- 01 — Satış raporu: SRP + OCP ile yeniden düzenleme ★★☆
- 02 — Kütüphane ödünç işlemleri: DIP + LSP ★★☆

15 İleri okuma

- Robert C. Martin, "The Principles of OOD" ve *Clean Architecture* (2017), III. bölüm — SOLID adlarının kaynağı.
- Barbara Liskov ve Jeannette Wing, "A Behavioral Notion of Subtyping" (1994).
- Gamma, Helm, Johnson, Vlissides, *Design Patterns* (1994), 1. bölüm — "Sınıf kalıtımı yerine nesne bileşimini tercih edin".
- JEP 513 — [Flexible Constructor Bodies](#)
- Mermaid — [sınıf diyagramları](#) · [sıralama diyagramları](#)
- Terimlerin Türkçesi: [docs/glossary.md](#)